# Python Functions

By PrudviKrishna

# Python Functions

**Introduction to Functions:**

**Definition**: A function is a block of code which performs a specific task. It helps to modularize code and makes it reusable.

**Why Use Functions?**

Code reuse, Modularity , Simplification of complex problems, Improved readability and maintainability.

**Syntax:**

**Defining the function**

```
def function_name(parameters):
    """
    Optional docstring (to describe the function).
    """
    # Body of the function
    return expression  # Optional
```

**Calling function**

```
function_name(arguments)
```

# Parameters and Arguments

**Parameters**: Variables listed inside the parentheses in the function definition.

**Arguments**: Values passed to the function when it is called.

**Default Arguments**

Default values can be assigned to parameters, making them optional during the function call.

**Example:**

```
def greet(name, message="Hello!"):
    print(f"{message} {name}")

greet("Alice")  # Uses default value for message
greet("Alice", "Good morning")  # Overwrites the default
```

# Variable Scope

**Local Scope**: Variables defined inside a function are local to that function and cannot be accessed outside of it.

**Global Scope**: Variables defined outside any function can be accessed globally.

**Example:**

```
global_var = 10

def modify_var():
    local_var = 5  # Local variable
    print(global_var)  # Accessing global variable

def useglobal_var():
    print(global_var)
```

# *args and **kwargs

***args**: Allows a function to accept any number of positional arguments as a tuple.

****kwargs**: Allows a function to accept any number of keyword arguments as a dictionary.

**Example:**

```
def multiply(*args):
    result = 1
    for num in args:
        result *= num
    return result

def display_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print(multiply(2, 3, 4))  # Output: 24
display_info(name='Alice', age=25, city='New York')
```

# Recursive Functions

**Recursive Functions**

A function can call itself recursively.

Useful for problems that can be divided into smaller sub-problems.

**Example (Factorial using recursion):**

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))  # Output: 120
```

# Higher-Order Functions

**Higher-Order Functions**

Functions that can take other functions as arguments or return functions as results.

**Example:**

```
def apply_func(func, value):
    return func(value)

def square(x):
    return x * x

print(apply_func(square, 4))  # Output: 16
```

# Built-in Functions

Some useful Python built-in functions:

len(): Returns the length of an object.
type(): Returns the type of an object.
map(): Applies a function to all items in an iterable.
filter(): Filters items in an iterable based on a function.

# Decorators

**Decorators** allow you to modify or extend the behavior of functions without changing their code.

```
@addition_functionality
def my_function(parameters):
    # logic
```

i.e. my_function = addition_functionality(my_function)

```
def addition_functionality(func):
    def wrapper(parameters):
        # update parameter or additional logic
        return func(updated parameter)
    return wrapper
```

# Naming Convention

| Identifier Type | Naming Convention | Example |
|---|---|---|
| Variable Names | lowercase_with_underscores | user_name |
| Function Names | lowercase_with_underscores | calculate_total() |
| Class Names | PascalCode | UserProfile |
| Constants | UPPERCASE_WITH_UNDERSCORES | MAX_CONNECTIONS |
| Modules | lowercase_with_underscores | my_module.py |
| Packages | lowercase_with_underscores | my_package |

# Thank you